PROBLEM 1 • Basic → Intermediate

Membership Field Reach Checker

Problem Statement

PageTurner Library's IT team wants the same kind of pre-compile linter MediTrack built, but for their own LibraryMember class — and with one extra feature: a breakdown of results grouped by modifier, not just a flat total, so the report can highlight exactly which modifier is causing the most rejected attempts.

Design AccessChecker with a single-attempt classifier, and design LibraryMember itself using the four access levels correctly for the reasons given below.

Requirements

- LibraryMember's membershipPin must be inaccessible from any class other than LibraryMember itself.
- branchCode must be reachable only by classes in the same package as LibraryMember.
- finesOwed must be reachable by the same package (its full cross-package/subclass behavior is covered in Problem 2).
- displayName must be reachable from anywhere.
- classifyAccess(...) must decide its answer purely from the real Java visibility rules, not from hardcoded field names.

Function Signature(s)

```
static String classifyAccess(String fieldModifier, String accessorContext)
static String summarizeByModifier(String[][] attempts)   // groups results per modifier, not as one flat total
```

Examples

Input	Output
`classifyAccess("private", "SAME_CLASS")`	`"ALLOWED"`

Input	Output
`classifyAccess("protected", "DIFFERENT_PACKAGE")`	`"DENIED"`

Input	Output
`summarizeByModifier({ {"private","SAME_CLASS"}, {"private","SAME_PACKAGE"}, {"default","SAME_PACKAGE"}, {"default","DIFFERENT_PACKAGE"}, {"protected","SAME_PACKAGE"}, {"protected","SAME_CLASS"}, {"public","DIFFERENT_PACKAGE"} })`	`"private: 1 allowed / 1 denied | default: 1 allowed / 1 denied | protected: 2 allowed / 0 denied | public: 1 allowed / 0 denied"`

Explanation: *Every modifier appears in the summary even when every one of its attempts happened to be allowed or denied — the grouping is by modifier, independent of how the results happen to split.*

> **HINT** Build the grouping around the four modifiers themselves, not around whatever modifiers happen to appear in a given batch — a modifier with zero attempts still shouldn't crash your summary if you're asked to extend this later.

CONSTRAINTS fieldModifier ∈ {"private","default","protected","public"}. accessorContext ∈ {"SAME_CLASS","SAME_PACKAGE","DIFFERENT_PACKAGE"}. Batch size up to 500 entries.

Key Concepts Tested: *private/default/protected/public visibility rules, per-category aggregation, designing a class with intentionally chosen access levels per field.*

PROBLEM 2 • Intermediate

Reference Desk Subclass Reach

Problem Statement

PageTurner's premium-membership module (a different package) extends LibraryMember to add loyalty perks, and the branch's reference desk keeps hitting confusing compile errors around finesOwed. Extend AccessChecker to handle the subclass-in-a-different-package rule precisely — including the version that catches almost everyone off guard.

This time, instead of a batch summary, build a method that finds the very first attempt in an ordered list that would actually be denied — useful for a linter that wants to stop and report the first real problem, not just a final tally.

Requirements

- Add accessorContext values SUBCLASS_DIFFERENT_PACKAGE_OWN_TYPE and SUBCLASS_DIFFERENT_PACKAGE_PARENT_TYPE.
- private and default must remain fully DENIED for both new contexts.
- protected must be ALLOWED for SUBCLASS_DIFFERENT_PACKAGE_OWN_TYPE, but DENIED for SUBCLASS_DIFFERENT_PACKAGE_PARENT_TYPE.
- firstDeniedAttempt(...) must scan attempts strictly in order and stop at the first one that would be denied — it must not simply reuse a batch-counting approach and then search the results afterward.

Function Signature(s)

```
static String classifyAccess(String fieldModifier, String accessorContext)
static String firstDeniedAttempt(String[][] attempts)   // returns a description, or "None Denied"
```

Examples

Input	Output

`firstDeniedAttempt({`
 	
`{"public","SUBCLASS_DIFFERENT_PACKAGE_PARENT_TYPE"},`
 	
`{"protected","SUBCLASS_DIFFERENT_PACKAGE_PARENT_TYPE"},`
 	
`{"protected","SUBCLASS_DIFFERENT_PACKAGE_OWN_TYPE"}`
`})`
	`"protected via SUBCLASS_DIFFERENT_PACKAGE_PARENT_TYPE (attempt #2)"`

Explanation: *The first attempt (public) is allowed, so scanning continues; the second attempt is where the very first denial actually occurs, and that's what gets reported — not the third one, even though it's also worth noting.*

Input	Output

`firstDeniedAttempt({`
 	
`{"public","SUBCLASS_DIFFERENT_PACKAGE_OWN_TYPE"},`
 	
`{"protected","SUBCLASS_DIFFERENT_PACKAGE_OWN_TYPE"}`
`})`
	`"None Denied"`

HINT Resist the temptation to build the full result list first and then search it — the whole point of "first denied" is that you should be able to stop scanning the moment you find it, without processing anything after it.

CONSTRAINTS Same modifier set as Problem 1, now with 5 possible accessorContext values. Batch size up to 500 entries.

Key Concepts Tested: *protected's genuinely subtle cross-package rule, compile-time vs. runtime typing, early-exit scanning as a distinct aggregation style from batch counting.*

PROBLEM 3 • Intermediate

Book Copy Circulation Guard

Problem Statement

A previous version of BookInventory let copiesAvailable be modified directly from anywhere, and a bug once pushed it to -1 during a busy exam-week rush — the catalog showed a book as available when every physical copy was already checked out. Redesign BookInventory so copiesAvailable can only ever move through validated methods.

checkOut() must never let the count go negative, and checkIn() must never let it exceed the total number of copies the branch actually owns — both invalid transitions should be rejected silently, without crashing anything.

Requirements

- copiesTotal and copiesAvailable must both be private.
- checkOut() must be rejected (with no state change) if no copies are currently available.
- checkIn() must be rejected (with no state change) if the inventory is already at full capacity — there is nothing to "return."
- There must be no way, from outside the class, to set copiesAvailable directly to an arbitrary value.

Function Signature(s)

```
BookInventory(int copiesTotal)
void checkOut()
void checkIn()
int getCopiesAvailable()
```

Examples

Input	Output

`BookInventory b = new BookInventory(3);`
`b.checkOut(); b.checkOut(); b.checkOut();`
`b.checkOut();  // 4th attempt`
`b.getCopiesAvailable()`
	`0`

Explanation: *The 4th checkOut() call is silently rejected — there's nothing left to check out, and the count must never go negative.*

Input	Output

`b.checkIn(); b.checkIn(); b.checkIn();`
`b.checkIn();  // 4th attempt`
`b.getCopiesAvailable()`
	`3`

Explanation: *The inventory is already back to full (3 of 3) after the third check-in — the 4th call has nothing genuine to return, and must be rejected rather than pushing the count past the total.*

HINT checkIn() needs to know not just "is copiesAvailable less than copiesTotal" in the abstract, but specifically whether this call represents a real book actually coming back — think about what invariant must hold between copiesAvailable and copiesTotal at every moment, not just at the boundaries.

CONSTRAINTS copiesTotal is a positive integer up to 500. Calls are processed one at a time, in the order they arrive.

Key Concepts Tested: *private fields with validated public methods ("one door in, one door out"), boundary-safe state transitions, silent-rejection validation.*

PROBLEM 4 • Intermediate → Advanced

LibraryMember JavaBean & Security Answer Property

Problem Statement

PageTurner's self-service kiosks use a JavaBean-scanning framework to auto-generate the membership signup form, exactly like MediTrack's dashboard. Build LibraryMember as a fully compliant JavaBean, with the same two unusual property rules applied to a different pair of fields.

membershipId must behave as write-once, even though a public setter genuinely exists to satisfy the framework's scan. securityAnswer (used for "forgot my card" verification at the desk) must be genuinely write-only — settable, but never retrievable again by any method on the class, in any form.

Requirements

- LibraryMember must have a public no-argument constructor.
- Every ordinary field needs a matching getX()/setX() pair, named to the exact JavaBean convention.
- The premium-membership status field must use isPremiumMember(), not getPremiumMember().
- setMembershipId(String id) must only take effect on its first call; every later call must be silently ignored.
- setSecurityAnswer(String answer) must store only a one-way transformed value, and no getter for it may exist anywhere on the class.

Function Signature(s)

```
public LibraryMember()
String getMembershipId();  void setMembershipId(String id)
String getName();  void setName(String name)
boolean isPremiumMember();  void setPremiumMember(boolean premium)
void setSecurityAnswer(String answer)   // no matching getter exists anywhere
```

Examples

Input	Output

`LibraryMember m = new LibraryMember();`
`m.setMembershipId("LIB-8841");`
`m.setName("Priya Nair");`
`m.setPremiumMember(true);`
`m.getMembershipId()`
	`"LIB-8841"`

Input	Output
`m.setMembershipId("FAKE-0000");` `m.getMembershipId()`	`"LIB-8841"`

Explanation: *Just like Problem 4 of the mentor set, the second call is silently ignored — write-once, built on top of a setter the framework requires to exist.*

Input	Output
`m.isPremiumMember()`	`true`

Input	Output
`m.setSecurityAnswer("BlueMountain");`	`(no observable output – no method anywhere on the class can retrieve this value again)`

HINT "Write-once" and "no getter exists" are two different protective patterns living side by side in the same class here — make sure you don't accidentally apply the wrong one to the wrong field.

CONSTRAINTS securityAnswer's stored transformation only needs to be deterministic and one-way — no real cryptographic library is required.

Key Concepts Tested: *JavaBean getX/setX/isX naming convention, the public no-argument constructor requirement, a write-once property built on a technically-required setter, a true write-only property.*

PROBLEM 5 • Advanced

Immutable Loan Receipt & Nightly Circulation Ledger

Problem Statement

Every time a member checks out books, PageTurner issues a LoanReceipt that must never change once created — it's the record used to resolve disputes about what was borrowed and when. Build LoanReceipt as a genuinely immutable class, proven immune to the same array-mutation trap covered in the mentor set's capstone.

Then build a nightly circulation processor that reconciles the day's receipts, including a Reference-Only variant for books that never leave the building (and which settle differently at night), while tolerating null placeholder entries from a flaky scanner feed without ever crashing the whole run.

Requirements

- LoanReceipt's fields must all be final, and the class itself must be declared final.
- The constructor must defensively copy the bookIds array on the way in, and getBookIds() must defensively copy again on the way out.
- Correcting a receipt (e.g., a mis-scanned book ID) must be done through a with-style method that returns a brand-new object, leaving the original untouched.
- CirculationLedger must set up any one-time, class-level state (e.g., a shared branch code) using a static block.
- The nightly processor must use instanceof to settle a ReferenceOnlyLoanReceipt differently from a regular one, and must never throw an uncaught exception because of a null entry.

Function Signature(s)

```
public LoanReceipt(String memberId, String[] bookIds)
String[] getBookIds()
LoanReceipt withCorrectedBookId(int index, String newId)
public ReferenceOnlyLoanReceipt(String memberId, String[] bookIds, String roomNumber)   // extends
LoanReceipt
static String processNightlyCirculation(LoanReceipt[] receipts)
```

Examples

Input	Output

`LoanReceipt r = new LoanReceipt("LIB-8841", new String[]{"BK-100","BK-101"});`
`String[] ids = r.getBookIds();`
`ids[0] = "HACKED";`
`r.getBookIds()[0]`
	`"BK-100"`

Explanation: *Mutating the returned array must never reach LoanReceipt's real internal data — the same defensive-copy discipline as the mentor set's capstone, applied here from scratch.*

Input	Output

`LoanReceipt corrected = r.withCorrectedBookId(1, "BK-102");`
`r.getBookIds()`
`corrected.getBookIds()`
	`["BK-100", "BK-101"]`
	`["BK-100", "BK-102"]`

Input	Output
`processNightlyCirculation({ new ReferenceOnlyLoanReceipt("LIB-001", new String[]{"BK-200"}, "Reading Room 3"), null, new LoanReceipt("LIB-002", new String[]{"BK-201"}) })`	`"2 processed | 1 null skipped | 1 reference-only | 1 regular"`

HINT Every array field needs defending in both directions independently — proving the constructor copies correctly says nothing about whether the getter does too, and vice versa. Test both directions separately.

CONSTRAINTS bookIds length up to 20. Batch size up to 5,000. processNightlyCirculation should complete in a single pass with O(1) extra space beyond a handful of running counters.

Key Concepts Tested: *True immutability (final fields + final class + defensive copying in and out), the "wither" pattern for simulating change, static initialization blocks, instanceof-based polymorphic batch dispatch, null-safety in aggregation — every concept from this document, combined.*